

Reporte de rendimiento del programa Calor2D

David Turcio Maya
david.turcio@correo.nucleares.unam.mx

ABSTRACT

Se presentan los resultados de aceleración y eficiencia obtenidos al correr un código en paralelo escrito para resolver la ecuación diferencial de calor en el equilibrio térmico ocupando 12 núcleos. Se realiza una comparación entre la ejecución del código en un ordenador personal y el cluster del Laboratorio de Modelación de Datos (LAMOD) cede Instituto de Ciencias Nucleares (ICN) de la UNAM.

INTRODUCCIÓN

El programa Calor2D simula la distribución de la temperatura sobre una malla cuadrada (condiciones de iniciales, condiciones de frontera y discretización de la malla se hablarán con mayor detalle en la sección teórica del siguiente reporte) suponiendo que se ha llegado al equilibrio térmico. Hace uso de la ecuación del calor descrita por la ecuación 1 cuando se tiene un estado de equilibrio térmico y resuelve esta ecuación diferencial parcial mediante el uso de la aproximación numérica para las derivadas al igual que una discretización de la malla.

$$\frac{\partial T}{\partial t} = \alpha \nabla^2 T \quad (1)$$

Originalmente el programa fue desarrollada por el Dr. Juan Carlos Cajas García para ejecutarse de manera secuencial. En la clase se trabajo con distintas maneras de paralelizar este código (mediante la API de OpenMP y OpenACC). En el siguiente trabajo se muestran las modificaciones realizadas al código para poder hacer uso de la API de OpenMP para ejecutar el programa de manera paralela. Se muestran los resultados de los cálculos realizados por el código mediante su visualización en ParaView y se muestra la aceleración y eficiencia del código al aumentar la cantidad de hilos para paralelizar las operaciones en los núcleos de un ordenador personal al igual que en el cluster LAMOD cede ICN de la UNAM.

TEORÍA

Ecuación de Calor

La ecuación de calor (que se puede ver en la ecuación 1) nos menciona que la cantidad de calor absorbido en un volumen por unidad de tiempo debe ser igual al flujo total de calor que pasa por este volumen a través de su superficie.

Cuando se llega a un estado de equilibrio, tenemos que la ecuación de la temperatura ya no va a variar con el tiempo, por lo que el programa calor2D la ecuación de calor se reduce a la siguiente ecuación de Laplace 2.

$$\nabla^2 T(x, y) = 0 \quad (2)$$

Aproximación para las segundas derivadas y discretización de la malla

El código calor2D hace uso de la ecuación diferencial 2 para obtener la distribución de temperatura dentro de una malla con condiciones de frontera en donde la temperatura es fija (se hablará más sobre las condiciones de frontera en el capítulo siguiente). Una manera muy común de realizar un programa que resuelva una ecuación diferencial de Laplace es ocupar que nuestro dominio este discretizado, para el caso de un código de una dimensión nuestra malla discretizada nos daría una serie de puntos N separados por una distancia h y para poder resolver la ecuación de Laplace se hace uso de la serie de Taylor 7 para poder obtener una aproximación de primer orden del valor de la segunda derivada evaluada en un punto x de la malla.

$$f(x + h) = f(x) + f'(x)h + \frac{1}{2}f''(x)h^2 + O(h^3) \quad (3)$$

$$f(x - h) = f(x) - f'(x)h + \frac{1}{2}f''(x)h^2 + O(h^3) \quad (4)$$

$$\text{sumando ambas ecuaciones:} \quad (5)$$

$$f(x + h) + f(x - h) = 2f(x) + f''(x)h^2 + O(h^3) \quad (6)$$

$$f''(x) = \frac{f(x + h) - 2f(x) + f(x - h)}{h^2} + O(h) \quad (7)$$

Debido a que la siguiente aproximación es de primer orden, en el dominio de nuestra malla se deben tener varios puntos para poder tener una buena aproximación. Para la ecuación de laplace, la segunda derivada se hace cero y nuestra aproximación de la segunda derivada se puede reducir a la ecuación 8.

$$f''(x) \approx f(x + h) - 2f(x) + f(x - h) = 0 \quad (8)$$

Para el caso bidimensional, se puede ocupar el mismo procedimiento de la aproximación por medio de la series de Taylor para cada una de las derivadas parciales, dando lugar a la siguiente aproximación.

$$\nabla^2 f(x, y) \approx \frac{f(x + h_x, y) + f(x - h_x, y)}{h_x^2} + \frac{f(x, y + h_y) + f(x, y - h_y)}{h_y^2} - f(x, y) \left(\frac{2}{h_x^2} + \frac{2}{h_y^2} \right) = 0 \quad (9)$$

Entre la ecuaci  n 8 y 9 podemos notar que hay una gran diferencia de complejidad, mientras que la ecuaci  n 8 se puede simplificar a un sistema de $N \times N$ ecuaciones lineales. Las ecuaciones 9 se simplificar  n a $(N_x \times N_x)(N_y \times N_y)$ ecuaciones lineales. El procedimiento que ocupa el programa para simplificar este problema implica resolver primero de manera lineal las ecuaciones que se encuentran en las filas de la malla y de esos resultados luego resolver para las columnas de la malla.

M  tricas de rendimiento

Debido a que el programa se puede correr de manera paralela especificando la cantidad de n  cleos mediante el comando `export OMP_NUM_THREADS`, es relevante para nosotros saber como es el rendimiento del programa a comparaci  n de su contraparte secuencial. Para lo cu  l se hace uso de los siguientes conceptos:

1. Aceleraci  n: Se le conoce como aceleraci  n al cociente: $S = \frac{t_{seq}}{t_{par}}$. Donde S es la aceleraci  n (se marca con esta letra debido a que en ingl  s se conoce el concepto como **Speed up**), t_{seq} es el tiempo de ejecuci  n secuencial (sin correr en paralelo) y t_{par} el tiempo de ejecuci  n paralelo.
2. Eficiencia: Para calcular la eficiencia se tiene que $E = \frac{S}{N}$ donde N es el n  mero de n  cleos empleados al correr el programa en paralelo.

Al programar en paralelo se espera que el el tiempo de ejecuci  n secuencial sea mayor o igual (en el caso de que se este ocupando un n  cleo) al tiempo de ejecuci  n paralelo.

PROGRAMA CALOR2D

El programa calor2D (anexado al final de este reporte) se encuentra escrito en lenguaje FORTRAN 90 y tiene como finalidad obtener el estado de equilibrio t  rmico haciendo uso de la ecuaci  n de calor (debido al equilibrio t  rmico la ecuaci  n se reduce a la de Laplace). El procedimiento por orden de escritura que sigue para resolver el problema es el siguiente:

1. Declaraci  n de variables e inicializaci  n de variables.
2. Definici  n de condiciones de frontera: Para el programa se ocuparon condiciones de frontera de Dirichlet en donde dos fronteras (la columna 1 y rengl  n 1) se mantienen a un valor de 1 mientras que el resto de las fronteras a un valor de cero.
3. Bucle de iteraciones: Dentro de este bucle se realizan los barridos en las filas y columnas de la matriz y mediante la ayuda de la rutina externa tridiagonal.f90 resuelve la ecuaci  n de Laplace. De igual manera, se tiene un bucle adicional de convergencia que funciona para detener el bucle una vez pasado a un criterio de tolerancia especificado.
4. Post-procesamiento y salida de resultados: Mediante el uso de un programa externo mod_utiles.f90 los resultados son procesados y exportados en un archivo .vtk para ser visualizados en ParaView.

Implementación de OpenMP en el programa calor2D

OpenMP se ocupa dentro del programa para paralelizar ciertos bucles (anexado al final de este reporte) haciendo uso de los siguientes comandos

Listing 1: Paralelización de bloque con OpenMP

```
!$omp parallel
... Bloque a paralelizar
!$omp end parallel
```

Listing 2: Paralelización de bucle con OpenMP

```
!$omp parallel do
... Bucle a paralelizar
!$omp end parallel do
```

Cabe resaltar que parte del análisis realizado en este reporte involucra la cantidad de bucles que se paralelizan con OpenMP.

RESULTADOS

El código se corrió para una malla de 128 x 128 y los resultados obtenidos fueron guardados en un archivo denominado como *salida.vtk* con la finalidad de ser analizados en el programa ParaView como se muestra en la imagen 1.

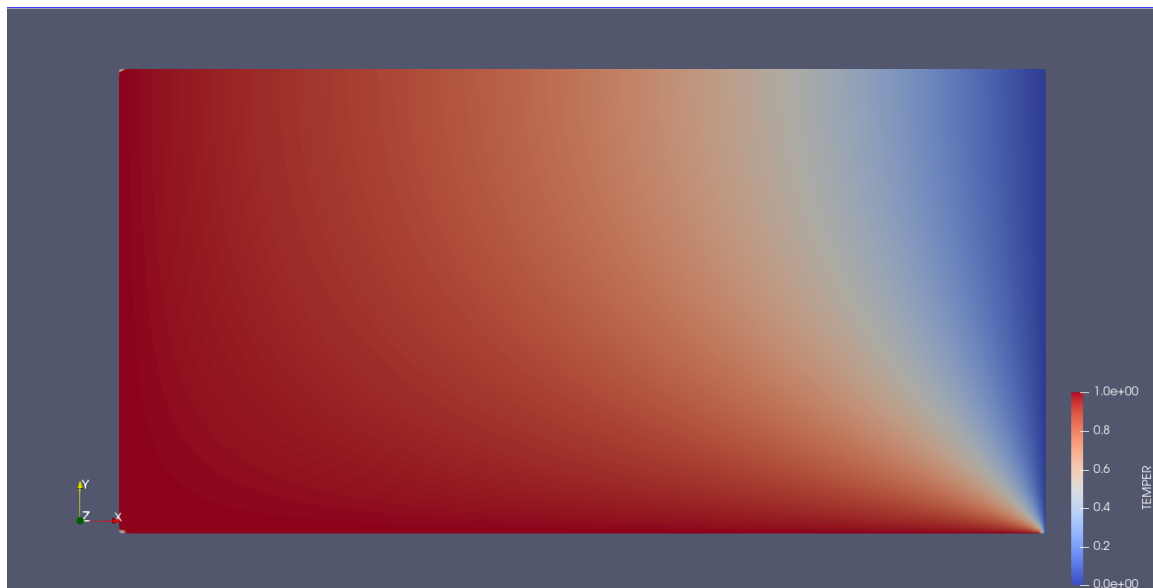


Figure 1: Procesamiento del archivo salida.vtk.

El programa se corrió dos veces en dos lugares. La primera corrida paraleliza los bucles del programa calor2D con la excepción del bucle de criterio de convergencia y la subrutina de residuo en mod_utiles.f90. Las localizaciones fueron en un ordenador personal de 12 nucleos

y en el cluster de LAMOD cede ICN con acceso a 48 n  cleos (aunque para poder realizar la comparaci  n solamente se ocuparon 12 de esos n  cleos, mayor informaci  n sobre el ordenador personal y el cluster se puede encontrar anexada al final de este documento). Los tiempos de ejecuci  n al igual que la cantidad de nodos y la cantidad de bucles modificados (denominados por *Antes de modificar* el cu  l incluye los bucles del programa calor2D con la excepci  n del bucle de criterio de convergencia y la subrutina de residuo en mod_utiles.f90 y *Despu  s de modificar* que paraleliza todos los bucles mencionados) se pueden encontrar en las siguientes tablas:

N��mero de Nodos	Antes de modificar (s)	Despu��s de modificar (s)
	(ordenador personal)	(ordenador personal)
secuencial	2.159	2.081
1	2.090	2.081
2	1.416	1.186
3	1.152	0.825
4	0.997	0.678
5	0.954	0.547
6	0.932	0.518
7	0.870	0.470
8	0.835	0.411
9	0.787	0.358
10	0.732	0.314
11	0.707	0.288
12	0.748	0.277

N��mero de Nodos	Antes de modificar (s)	Despu��s de modificar (s)
	(cluster)	(cluster)
secuencial	2.405	2.949
1	2.404	3.050
2	1.309	1.518
3	1.052	1.016
4	0.878	0.783
5	0.771	0.656
6	0.732	0.557
7	0.731	0.503
8	0.739	0.427
9	0.756	0.400
10	0.794	0.370
11	0.955	0.333
12	0.979	0.314

DISCUSI  N DE RESULTADOS Y CONCLUSIONES

El programa calor2D obtiene resultados (salida.vtk) que concuerdan con lo esperado al tratar con el problema del equilibrio t  rmico.

Para la aceleraci  n y eficiencia al paralelizar el c  digo en los cuatro casos mencionados

anteriormente y con ayuda de las tablas del resultados podemos obtener las siguientes gráficas.

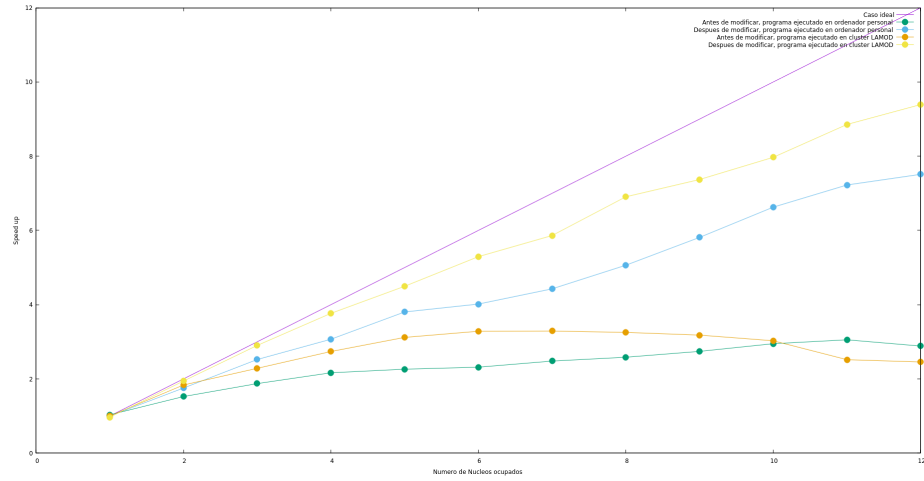


Figure 2: Gráfica comparando aceleración en los cuatro casos mencionados.

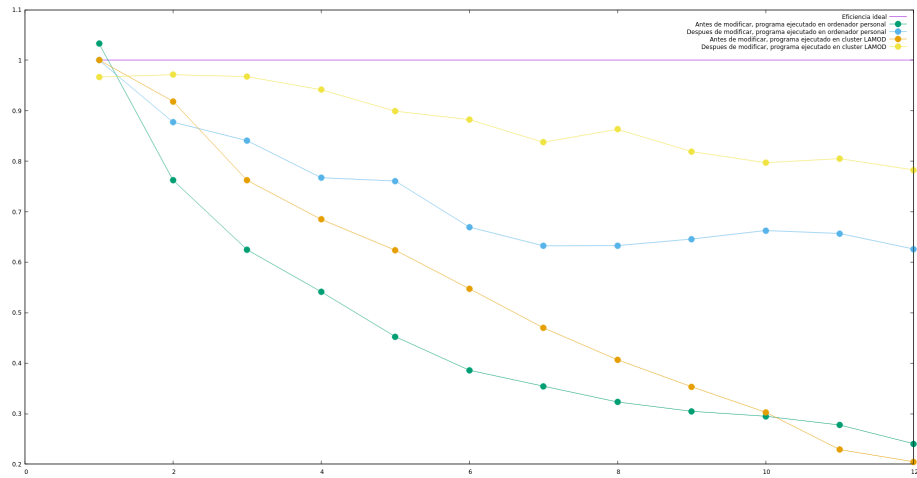


Figure 3: Gráfica comparando eficiencia en los cuatro casos mencionados.

Podemos observar claramente que la eficiencia y aceleración del programa mejoran considerablemente al paralelizar todos los bucles mencionados. También cabe mencionar que la ejecución del programa en el cluster del LAMOD incremento la eficiencia y aceleración del programa, esto se debe a que a pesar de trabajar con la misma cantidad de núcleos en ambos casos, el cluster trabaja con más núcleos lógicos.

En la gráfica de eficiencia 3 podemos notar que para el núcleo 1 las cuatro gráficas muestran un valor distinto de 1, esto se debe al hecho de que al programar en paralelo se realizaron operaciones adicionales que no se realizan si se corre en secuencial.

Finalmente se concluye que la eficiencia y aceleración del programa paralelizado va a depender de la cantidad de bucles paralelizados al igual que la cantidad de núcleos lógicos que disponga la computadora que se utilice para correr este código.

APPENDIX A

DATOS DE ORDENADOR PERSONAL Y DE CLUSTER

Datos de Ordenador personal:

Arquitectura:	x86_64
modo(s) de operación de las CPUs:	32-bit, 64-bit
Address sizes:	39 bits physical, 48 bits virtual
Orden de los bytes:	Little Endian
CPU(s):	12
Lista de la(s) CPU(s) en línea:	0-11
ID de fabricante:	GenuineIntel
Nombre del modelo:	Intel(R) Core(TM) i7-10750H CPU @ 2.60G Hz
Familia de CPU:	6
Modelo:	165
Hilo(s) de procesamiento por núcleo:	2
Núcleo(s) por socket:	6
Socket(s)	1
Revisión:	2
CPU MHz máx.:	5000.0000
CPU MHz mín.:	800.0000
BogoMIPS:	5199.98
Indicadores:	fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36 clflush h dts acpi mmx fxsr sse sse2 ss ht tm p be syscall nx pdpe1gb rdtscp lm constan t_tsc art arch_perfmon pebs bts rep_goo d nopl xtopology nonstop_tsc cpuid aper fmpervf pni pclmulqdq dtes64 monitor ds_ cpl vmx est tm2 ssse3 sdbg fma cx16 xtp r pdcm pcid sse4_1 sse4_2 x2apic movbe popcnt tsc_deadline_timer aes xsave avx f16c rdrand lahf_lm abm 3dnowprefetch cpuid_fault epb ssbd ibrs ibpb stibp ib rs_enhanced tpr_shadow flexpriority ept vpid ept_ad fsgsbase tsc_adjust bmi1 a vx2 smep bmi2 erms invpcid mpx rdseed a dx smap clflushopt intel_pt xsaveopt xs avec xgetbv1 xsaves dtherm ida arat pln pts hwp hwp_notify hwp_act_window hwp_ epp vnmi pku ospke md_clear flush_l1d a rch_capabilities ibpb_exit_to_user
Virtualization features:	
Virtualización:	VT-x
Caches (sum of all):	

```

L1d:                                192 KiB (6 instances)
L1i:                                192 KiB (6 instances)
L2:                                 1.5 MiB (6 instances)
L3:                                 12 MiB (1 instance)
NUMA:
  Modo(s) NUMA:                      1
  CPU(s) del nodo NUMA 0:             0-11
Vulnerabilities:
  Gather data sampling:               Mitigation; Microcode
  Indirect target selection:          Mitigation; Aligned branch/return thunks
  Itlb multihit:                     KVM: Mitigation: VMX disabled
  L1tf:                              Not affected
  Mds:                               Not affected
  Meltdown:                          Not affected
  Mmio stale data:                   Mitigation; Clear CPU buffers; SMT vulnerable
  Reg file data sampling:             Not affected
  Retbleed:                          Mitigation; Enhanced IBRS
  Spec rstack overflow:              Not affected
  Spec store bypass:                 Mitigation; Speculative Store Bypass disabled via prctl
  Spectre v1:                        Mitigation; usercopy/swapgs barriers and __user pointer sanitization
  Spectre v2:                        Mitigation; Enhanced / Automatic IBRS; IBPB conditional; PBR SB-eIBRS SW sequence; BHI SW loop, KVM SW loop
  Srbds:                             Mitigation; Microcode
  Tsa:                               Not affected
  Tsx async abort:                   Not affected
  Vmscape:                           Mitigation; IBPB before exit to userspace

```

Datos de Cluster

```

Architecture:                       x86_64
CPU op-mode(s):                     32-bit, 64-bit
Address sizes:                       46 bits physical, 48 bits virtual
Byte Order:                          Little Endian
CPU(s):                              48
On-line CPU(s) list:                 0-47
Vendor ID:                           GenuineIntel
Model name:                          Intel(R) Xeon(R) CPU E5-2690 v3 @ 2.60GHz
CPU family:                          6
Model:                               63
Thread(s) per core:                  2
Core(s) per socket:                  12

```



```

Socket(s):                2
Stepping:                 2
CPU(s) scaling MHz:      95%
CPU max MHz:              3500.0000
CPU min MHz:              1200.0000
BogoMIPS:                 5199.93
Flags:                    fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pg
                          e mca cmov pat pse36 clflush dts acpi mmx fxsr sse
                          sse2 ss ht tm pbe syscall nx pdpe1gb rdtscp lm cons
                          tant_tsc arch_perfmon pebs bts rep_good nopl xtopol
                          ogy nonstop_tsc cpuid aperfmperf pni pclmulqdq dtes
                          64 monitor ds_cpl vmx smx est tm2 ssse3 sdbg fma cx
                          16 xtpr pdcm pcid dca sse4_1 sse4_2 x2apic movbe po
                          pcnt tsc_deadline_timer aes xsave avx f16c rdrand l
                          ahf_lm abm cpuid_fault epb pti intel_ppin ssbd ibrs
                          ibpb stibp tpr_shadow flexpriority ept vpid ept_ad
                          fsgsbase tsc_adjust bmi1 avx2 smep bmi2 erms invpc
                          id cqm xsaveopt cqm_llc cqm_occup_llc dtherm ida ar
                          at pln pts vnmi md_clear flush_lld

Virtualization features:
  Virtualization:         VT-x
Caches (sum of all):
  L1d:                    768 KiB (24 instances)
  L1i:                    768 KiB (24 instances)
  L2:                     6 MiB (24 instances)
  L3:                     60 MiB (2 instances)
NUMA:
  NUMA node(s):           2
  NUMA node0 CPU(s):      0-11,24-35
  NUMA node1 CPU(s):      12-23,36-47
Vulnerabilities:
  Gather data sampling:    Not affected
  Indirect target selection: Not affected
  Itlb multihit:          KVM: Mitigation: Split huge pages
  L1tf:                   Mitigation; PTE Inversion; VMX conditional cache fl
                          ushes, SMT vulnerable
  Mds:                    Mitigation; Clear CPU buffers; SMT vulnerable
  Meltdown:               Mitigation; PTI
  Mmio stale data:        Mitigation; Clear CPU buffers; SMT vulnerable
  Reg file data sampling:  Not affected
  Retbleed:               Not affected
  Spec rstack overflow:    Not affected
  Spec store bypass:      Mitigation; Speculative Store Bypass disabled via p
                          rctl
  Spectre v1:             Mitigation; usercopy/swapgs barriers and __user poi
                          nter sanitization
  Spectre v2:             Mitigation; Retpolines; IBPB conditional; IBRS_FW;
                          STIBP conditional; RSB filling; PBRSE-eIBRS Not aff

```

	ected; BHI Not affected
Srbds:	Not affected
Tsa:	Not affected
Tsx async abort:	Not affected
Vmscape:	Mitigation; IBPB before exit to userspace

APPENDIX B

PROGRAMA CALOR2D

Program Calor2D

```

!
use omp_lib
!
use utiles, only : postproceso_vtk
use utiles, only : residuo_temp
use utiles, only : nx, ny, itermax
!
Implicit none
!
! Declaracion de variables
!
! Iteradores y tamaño del problema
!
integer :: ii, jj, iter
!
! Variables del dominio computacional
!
double precision :: lx, ly, deltax, deltax
!
! Variable para el residuo de iteraciones, y tolerancia
!
double precision :: residuo, tolerancia, resid_u
!
! Variables del problema fisico
!
double precision :: xx(nx), yy(ny) ! variables de la malla
double precision :: tt(nx,ny,2)    ! vector de incognitas, 1 para la iteraci'on actual
                                   y 2 para la anterior
double precision :: resid_tt(nx,ny) ! vector de residuo
double precision :: tx(nx), ty(ny)  ! vectores de incognitas 1D
double precision :: rx(nx),ry(ny)   ! vector de resultados del s. ecuaciones
double precision :: cfx(ny,2),cfy(nx,2) ! vector de condiciones de frontera
!
double precision :: ax(nx), bx(nx), cx(nx) ! Variables para almacenar
!
                                   ! matriz tridiagonal sobredimensionada
!

```

```
double precision :: ay(ny), by(ny), cy(ny) ! Variables para almacenar
!                                     ! matriz tridiagonal sobredimensionada
!
! Variables de postproceso
!
character(48) :: archivo
!
! Tolerancia
!
tolerancia = 1d-4
!
! Dominio computacional
!
lx      = 10.d0
deltax  = lx/nx
ly      = 5.d0
deltay  = ly/ny
do jj = 1, ny
    yy(jj)=(jj-1)*deltay
end do
do ii = 1, nx
    xx(ii)=(ii-1)*deltax
end do
!
! Inicializacion de variables
!
! xx(:)  = 0.d0
ax(:)    = 0.d0
bx(:)    = 0.d0
cx(:)    = 0.d0
rx(:)    = 0.d0
ay(:)    = 0.d0
by(:)    = 0.d0
cy(:)    = 0.d0
ry(:)    = 0.d0
tt(:, :, :) = 0.d0
!
! Condiciones de frontera en direcci  n x
!
! Estos bucles pueden paralelizarse, pero hay que valorar bien la cantidad
! de trabajo que se va a compartir respecto al tiempo necesario para abrir
! los hilos paralelos.
!
do ii = 1, ny
    cfx(ii,1) = 1.d0
    cfx(ii,2) = 0.d0
end do
!
```

```

! Condiciones de frontera en direcci'on y
!
do jj = 1, nx
    cfy(jj,1) = 1.d0
    cfy(jj,2) = 0.d0
end do
!
bucle_iteraciones: do iter = 1, itermax
    !
    ! Inicializamos el valor de la iteraci'on anterior
    !
    ! $omp parallel do default(none) &
    ! $omp shared(tt)
    copia_anterior: do jj = 1, ny
do ii = 1, nx
tt(ii,jj,2) = tt(ii,jj,1)
end do
end do copia_anterior
! $omp end parallel do
    !
    !-----
    !
    ! Paralelizamos el barrido en la direcci'on y en bandas
    ! compuestas por grupos de l'ineas, observamos que si usamos pocas
    ! l'ineas tenemos una aceleraci'on pobre o ausente
    !
    !$omp parallel do default(none) &
    !$omp shared( deltax, deltay, tt, cfx) &
    !$omp private( ax, bx, cx, rx, tx, ii )
    barrido_y: do jj = 2, ny-1
        !
        ! Es posible combinar directivas de openmp, por ejemplo,
        ! si necesitamos una regi'on paralela unicamente para un bucle,
        ! podemos combinar "parallel" con "do"
        !
        ensambla_tri_x: do ii = 2, nx-1

            ax(ii) = 1.d0/(deltax*deltax)
            bx(ii) = -2.d0*(1.d0/(deltax*deltax)+ 1.d0/(deltay*deltay))
            cx(ii) = 1.d0/(deltax*deltax)
            rx(ii) = -1.d0/(deltay*deltay)*tt(ii,jj-1,1)-1.d0/(deltay*deltay)&
                & * tt(ii,jj+1,1)

        end do ensambla_tri_x
        !
        ! Impone cond. frontera
        !
        ax(1)      = 0.d0 ! no se usa en los c'alculos

```

```

    bx(1)      = 1.d0
    cx(1)      = 0.d0
    rx(1)      = cfx(jj,1)
    !
    ax(nx)     = 0.d0
    bx(nx)     = 1.d0
    cx(nx)     = 0.d0 ! no se usa en los c'alculos
    rx(nx)     = cfx(jj,2)
    !
    ! Resolver el problema algebraico
    !
    call tri(ax,bx,cx,rx,tx,nx)
    !
    ! Actualizar la temperatura de la placa
    !
    do ii = 1, nx

        tt(ii,jj,1) = tx(ii)

    end do
    !
end do barrido_y
!$omp end parallel do
!
!-----
!
! Paralelizamos el barrido en la direcci'on x en bandas
! compuestas por grupos de l'ineas, observamos que si usamos pocas
! l'ineas tenemos una aceleraci'on pobre o ausente
!
!$omp parallel do default(none) &
!$omp shared( deltax, deltay, tt, cfy) &
!$omp private( ay, by, cy, ry, ty )
barrido_x: do ii = 2, nx-1

    ensambla_tri_y: do jj = 2, ny-1

        ay(jj) = 1.d0/(deltay*deltay)
        by(jj) = -2.d0*(1.d0/(deltax*deltax)+ 1.d0/(deltay*deltay))
        cy(jj) = 1.d0/(deltay*deltay)
        ry(jj) = -1.d0/(deltax*deltax)*tt(ii-1,jj,1)-1.d0/(deltax*deltax)*&
            tt(ii+1,jj,1)

    end do ensambla_tri_y
    !
    ! Impone cond. frontera
    !
    ay(1)      = 0.d0 ! no se usa en los c'alculos

```

```

by(1)      = 1.d0
cy(1)      = 0.d0
ry(1)      = cfy(ii,1)
!
ay(ny)     =-1.d0
by(ny)     = 1.d0
cy(ny)     = 0.d0 ! no se usa en los c'alculos
ry(ny)     = cfy(ii,2)
!
! Resolver el problema algebraico
!
call tri(ay,by,cy,ry,ty,ny)
!
!
! Actualizar la temperatura de la placa
!
do jj = 1, ny

    tt(ii,jj,1) = ty(jj)

end do
!
end do barrido_x
!$omp end parallel do
!
! Criterio de convergencia
!
residuo = 0.d0
! $omp parallel do default(none) &
! $omp shared(tt,residuo)
do jj = 1, ny
    do ii = 1, nx
        residuo = residuo + (tt(ii,jj,1)-tt(ii,jj,2))*(tt(ii,jj,1)-tt(ii,jj,2))
    end do
end do
! $omp end parallel do
!
residuo = sqrt(residuo)
!
! write(*,*) "DEBUG: ", iter, residuo
!
if( residuo < tolerancia )exit
!
end do bucle_iteraciones
!
write(*,*) "Convergencia en ", iter, " iteraciones"
!
call residuo_temp( tt(1:nx,1:ny,1), deltax, deltay, resid_tt )

```

```
!  
! Aunque es muy tentador, no podemos paralelizar este bucle,  
! el archivo queda desordenado y gnuplot (y otros graficadores) no  
! los procesan bien.  
!  
archivo = 'salida.vtk'  
!  
call postproceso_vtk(xx,yy,tt(1:nx,1:ny,1), resid_tt ,archivo)  
!  
end Program Calor2D
```